Question Answering with a Custom Transformer Encoder and Generative Decoder

[GitHub] | [Weights – OneDrive] | [Web App]

NLP (AID 829) — Group Assignment | Group 18

S.No.	Name	Roll Number
1	Vishal Sriram	BT2024036
2	Harsha	BT2024106
3	Lohith	BT2024248
4	Munjikesh	BT2024247

Abstract

We built a SQuAD-based question answering system featuring two parallel tracks: an extractive span predictor and a custom generative answer decoder. Both tracks share a custom 12-layer Transformer encoder pretrained from scratch via masked language modeling. The extractive model was benchmarked against strong public baselines, confirming that our scratch-built encoder learns a competitive span-selection representation. For the generative track, we developed a bespoke 4-layer decoder optimized for concise answer generation. Evaluated on a shared validation subset, our custom decoder outperformed T5-small and Flan-T5-small, closely approaching T5-base.

1 Introduction

Machine reading comprehension requires a model to process a question and context passage, then return a precise answer. This project addressed the task through two distinct modes: **extractive QA** (predicting start and end token indices of the answer span) and **generative QA** (decoding a short answer token-by-token from the encoder states). Implementing both modes provided a rigorous test of the underlying encoder's representational capacity.

The pipeline executed in four sequential stages:

1. **Stage 1:** Pretrain a BERT-style encoder from scratch using masked language modeling on Wikipedia + C4.
2. **Stage 2:** Fine-tune the encoder for extractive QA on SQuAD v2.
3. **Stage 3:** Construct a generative encoder-decoder on top of the same encoder.
4. **Stage 4:** Expose both inference paths through a unified local demo.

Primary contribution. A clear engineering and research progression: a single custom-trained encoder powering both accurate extractive span prediction and competitive generative answer decoding without relying on off-the-shelf pretrained architectures.

2 Related Work

Extractive QA was revolutionized by BERT [1], which introduced bidirectional Transformer pretraining via masked language modeling. Our encoder architecture closely follows the BERT-base specification: a 12-layer Transformer stack with 768-dimensional hidden states. While robustly optimized variants like RoBERTa [2] achieve higher absolute performance on SQuAD [4], our project focused on the pedagogical value of pretraining this architecture from scratch.

In generative QA, unified text-to-text Transformers like T5 [3] dominate. We use T5-small and T5-base as external benchmarks, but our approach differs by grafting a lighter, bespoke 4-layer decoder onto our BERT-style encoder via a linear projection bridge. This hybrid design allowed strict control over decoding heuristics without inheriting the broad generation priors of massively pretrained seq2seq models.

3 System Architecture

The system was designed around a single shared encoder whose contextual representations power two structurally distinct inference heads. This unified backbone architecture reflects a deliberate engineering choice: rather than training separate models for extractive and generative QA, we exploit the fact that both tasks require the same fundamental capability — grounding token-level representations in the joint context of a question and a supporting passage.

3.1 Input Representation

Both inference paths begin with identical preprocessing. The question and context are concatenated using the standard BERT-style `[CLS] Q [SEP] C [SEP]` template. Three embedding matrices are summed to form the initial token representation: a token embedding ($|\mathcal{V}| \times 768$), a learned positional embedding (up to 512 positions), and a segment embedding that distinguishes question tokens (type 0) from context tokens (type 1). The summed embeddings are passed through layer normalization and dropout before entering the encoder stack.

3.2 Shared Transformer Encoder

The encoder is a 12-layer bidirectional Transformer. Each layer applies pre-layer-norm multi-head self-attention (12 heads, $d_k = d_v = 64$) followed by a two-layer feed-forward network with hidden dimension 3072 and GELU activation. Residual connections wrap both sub-layers. The full sequence output $\mathbf{H} \in R^{L \times 768}$ is passed to both downstream heads, where L is the padded sequence length (capped at 256 tokens with a doc stride of 64 for long contexts).

The pre-norm variant was chosen over the original post-norm BERT design because it yields more stable gradients during fine-tuning, particularly when the encoder is jointly optimized with the decoder at a lower learning rate.

3.3 Extractive Head

The extractive head is a single linear projection $\mathbf{W} \in R^{768 \times 2}$ applied independently to every encoder output token. This produces two scalar logits per position: s_i (start score) and e_i (end score). At inference time, the top-n_{best} start positions are enumerated and paired with all legal end positions to form candidate spans. Each span is scored by summing its start and end logits, and the highest-scoring valid span is returned as the extracted answer. For unanswerable questions (SQuAD v2 no-answer instances), the score of the `[CLS]` token logit serves as the null-answer score; if it exceeds the best span score by a tuned threshold τ, the model abstains.

3.4 Encoder–Decoder Bridge

Because the encoder operates in a 768-dimensional space while the decoder uses a 512-dimensional space, a learned linear projection $\mathbf{W}_{\text{bridge}} \in R^{768 \times 512}$ maps the full encoder memory $\mathbf{H}$ into the decoder's key–value space before each cross-attention layer. This projection is trained jointly with the decoder and allows the two modules to operate at different widths without any information bottleneck beyond what the linear map can express.

3.5 Generative Decoder

The generative decoder is a 4-layer autoregressive Transformer with hidden dimension 512 and 8 attention heads ($d_k = d_v = 64$). Each decoder block applies three sub-layers in sequence, each wrapped in a pre-norm residual:

1. **Masked self-attention.** Attends over all previously generated tokens using a causal mask, building up a representation of the partial answer prefix.
2. **Cross-attention.** Queries the projected encoder memory $\mathbf{W}_{\text{bridge}}\mathbf{H}$ to ground each decoding step in the source context, directly suppressing hallucination.
3. **Feed-forward network.** A two-layer FFN (hidden dim 2048, GELU) refines the attended representation before the next layer.

Token generation is initiated with a `[BOS]` token and terminated by `[EOS]` or the `max_new_tokens=12` hard cap, whichever comes first. Decoding uses beam search (`beam=4`) with a length penalty of 1.0 to discourage unnecessarily short outputs.

3.6 No-Answer Gating

Both heads implement a no-answer abstention mechanism, but using different signals. The extractive head uses a threshold on the `[CLS]` logit margin, while the generative head compares the token-level log-probability of the generated sequence against the log-probability of a fixed no-answer template string (*"unanswerable"*). If the template scores higher by a tuned margin δ, the model outputs *"Context has no answer"* rather than a generated span.

3.7 Design Rationale

The decision to share a single encoder across both heads rather than training two separate models offered three concrete advantages. First, it halved the pretraining compute required to obtain a strong contextual representation. Second, it enabled direct apples-to-apples comparison of extractive and generative decoding strategies on identical encoder representations. Third, the unified architecture made the demo deployment simpler: a single forward pass through the encoder feeds both heads, and the user can switch between modes without reloading weights.

The asymmetric width (768 encoder vs. 512 decoder) was a deliberate capacity trade-off. The encoder must model long bidirectional context over question-passage pairs of up to 256 tokens, requiring greater width. The decoder generates short answers (median 3–4 tokens in SQuAD), so a narrower 512-dimensional space is sufficient and reduces the decoder parameter count by roughly 40% compared to a width-matched design.

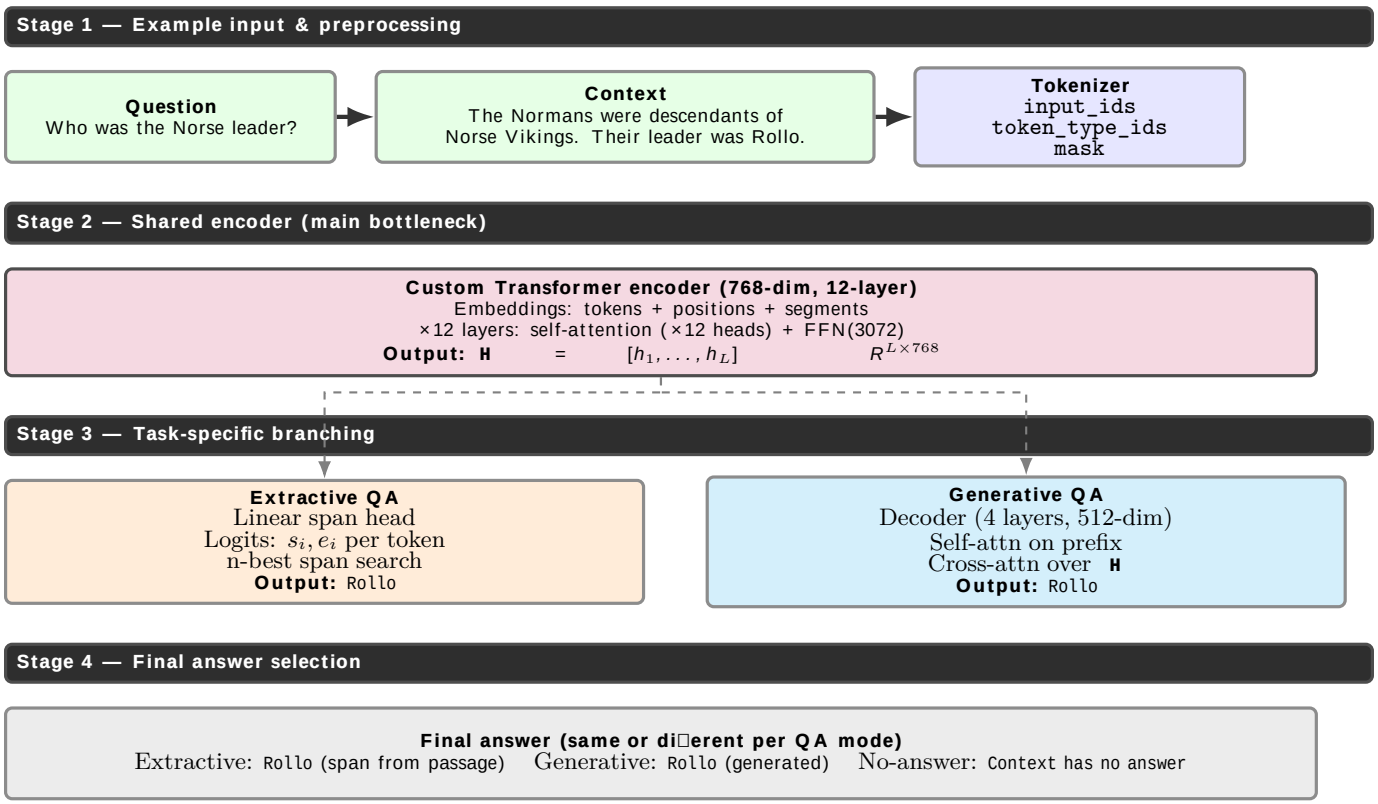

Figure 1: System architecture.

4 Method

4.1 Encoder Pretraining

The pipeline was grounded in a custom Transformer encoder pretrained from scratch using masked language modeling (MLM). 15% of input tokens were randomly masked using the 80/10/10 replacement strategy, and the model learned to predict the original tokens from surrounding context. Pretraining used an interleaved stream of Wikipedia (70%) and C4 (30%) data.

4.2 Extractive Span Prediction

For extractive QA, the pretrained encoder was equipped with a linear head that predicted start and end logits for each token position. At inference time, the system searched over the top n_{best} start/end pairs, validated spans against a maximum answer length constraint, and selected the highest-scoring valid span. For unanswerable questions, the CLS token logit was compared against the best span score via a tuned threshold.

4.3 Generative Decoder

The generative decoder was the main architectural contribution. Each of the four decoder blocks applied three operations in sequence:

1. Pre-norm masked self-attention on the generated prefix.
2. Residual cross-attention over the projected encoder memory.
3. Feed-forward refinement with residual connection.

This arrangement forced the decoder to stay anchored to the encoder's contextual evidence, reducing hallucination and sentence-copying behavior.

4.4 Decoding and No-Answer Gating

To produce concise outputs, we constrained decoding to greedy mode (`beam=1`), a strict `max_new_tokens=12` limit, and a length penalty of 0.4. For unanswerable questions, the system computed the log-probability of the generated answer and compared it against the log-probability of a fixed no-answer template. If the difference crossed a tuned threshold, the model abstained.

4.5 Loss Functions

We optimized three task-specific losses:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in M} \log p_\theta(x_i \mid x_{\setminus i}), \tag{1}$$

where $\mathcal{M}$ is the masked-token set (15%).

$$\mathcal{L}_{\text{ext}} = \frac{1}{2}\left[\text{CE}(\hat{y}^{start}, y^{start}) + \text{CE}(\hat{y}^{end}, y^{end})\right], \tag{2}$$

for extractive start/end supervision.

$$\mathcal{L}_{\text{gen}} = \text{CE}(\hat{y}_{1:T}, y_{1:T};\ \texttt{ignore_index} = -100,\ \ {}_{ls}), \tag{3}$$

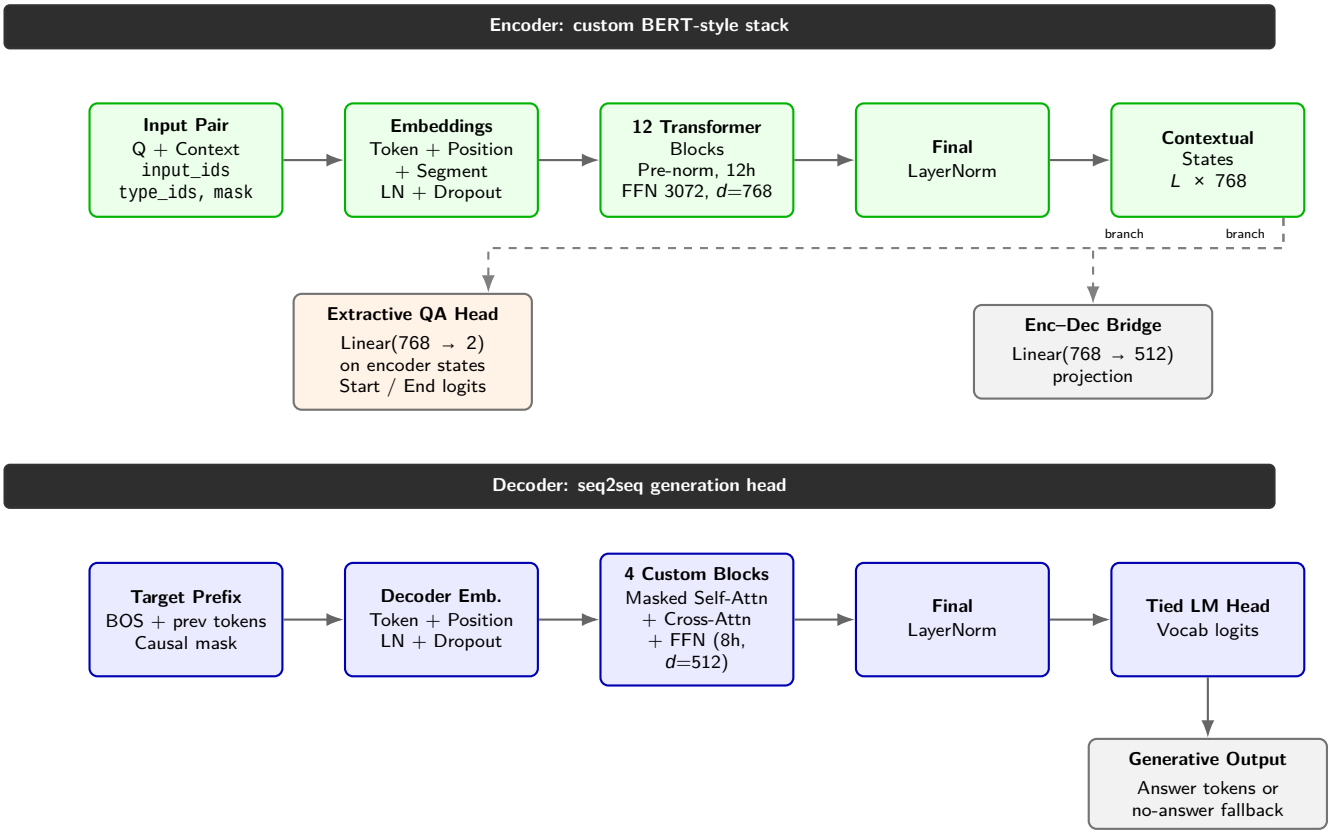

Figure 2: Detailed encoder and decoder architecture.

for teacher-forced autoregressive decoding with label smoothing $_{ls} = 0.05$.

5 Experimental Setup

Training was conducted on an RTX 4060 Ti GPU with fp16 mixed precision to accommodate the memory requirements of Transformer pretraining within 8 GB VRAM. This halved the activation memory footprint, allowing a batch size of 16 during the MLM pretraining phase while accelerating tensor core operations.

We implemented a robust checkpointing strategy with `nohup` wrappers for uninterrupted background execution. The training loop managed `latest.pt` and `best.pt` pointers, enabling clean resume from the exact step if a job was interrupted. All experiments used seed 42 for reproducibility.

Evaluation used the full 11,873-example validation set for final reported extractive metrics.

Table 1: Stage-wise training configuration.

Stage	Parameter	Setting
Stage 1: MLM Pre-training	Architecture	12 layers, 768 hidden, 12 heads, FFN 3072
	Batch / Accum	16 / 8 (e□. 128)
	LR / Schedule	1e-4, warmup 10% + cosine decay
	Precision	fp16/bf16 mixed
Stage 2: Extractive QA	Epochs / LR	3 / 5e-5
	Max length	256, doc stride 64
	No-answer	CLS null threshold sweep
Stage 3: Generative QA	Decoder	4 layers, 512 hidden, 8 heads
	Learning rates	3e-4 decoder, 8e-5 encoder
	Label smoothing	= 0.05
	Inference	greedy, max 12 tokens, lp 0.4

Table 2: Datasets and evaluation metrics used at each stage.

Stage	Dataset	Objective	Metrics
Pretrain	Wiki + C4	Masked language modeling	MLM loss
Extractive	SQuAD v2	Start/end span prediction	EM, F1
Generative	SQuAD v2	Token-by-token generation	EM, F1, ROUGE-L, BLEU